SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA14 **COURSE NAME: COMPILER DESIGN**

Name : Krishna Tejesh Reddy.Y
Reg no : 192411200

COURSE OUTCOME COVERED

CO3: *Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree.* *(BL4)*

Assessment Tool Weightage: 40%

Duration :60 Mins
On Class
QUESTIONS:5 **Total Marks:50**

Annexure A

EXPERIMENTS

Code of Conduct:

I Krishna Tejesh Reddy (192411200) _ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

EXPERIMENTS

Q1. Write a C program to implement a syntax-directed definition for evaluating arithmetic expressions using the following grammar:

Code ;

```c
#include <stdio.h>

int main()
{
    int num1 = 3;
    int num2 = 5;
    int num3 = 2;

    int T1, T2, E;

    printf("Syntax Directed Definition\n");
    printf("-------------------------\n");

    printf("F -> num : F.val = %d\n", num1);
    printf("F -> num : F.val = %d\n", num2);
    printf("F -> num : F.val = %d\n", num3);

    T2 = num2 * num3;

    printf("T -> T * F : T.val = %d * %d = %d\n",
           num2, num3, T2);

    T1 = num1;

    printf("T -> F : T.val = %d\n", T1);

    E = T1 + T2;

    printf("E -> E + T : E.val = %d + %d = %d\n",
           T1, T2, E);

    printf("\nFinal value of 3 + 5 * 2 = %d\n", E);

    return 0;
}
```

Output :

```
PS E:\Compiler design\CO3-AT1> gcc Q1.c -o Q1.exe
PS E:\Compiler design\CO3-AT1> .\Q1.exe
Syntax Directed Definition
--------------------------
F -> num : F.val = 3
F -> num : F.val = 5
F -> num : F.val = 2
T -> T * F : T.val = 5 * 2 = 10
T -> F : T.val = 3
E -> E + T : E.val = 3 + 10 = 13

Final value of 3 + 5 * 2 = 13
PS E:\Compiler design\CO3-AT1>
```

Q2. Write a C program to implement a simple type checker for arithmetic expressions using the following grammar:

Code :

```c
#include <stdio.h>
#include <string.h>

char* getType(char id[])
{
    if (strcmp(id, "a") == 0)
        return "int";

    if (strcmp(id, "b") == 0)
        return "int";

    if (strcmp(id, "c") == 0)
        return "float";

    return "unknown";
}

char* resultType(char type1[], char type2[])
{
    if (strcmp(type1, "float") == 0 ||
        strcmp(type2, "float") == 0)
        return "float";

    if (strcmp(type1, "int") == 0 &&
        strcmp(type2, "int") == 0)
        return "int";
```

```
    return "type error";
}

int main()
{
    char *aType;
    char *bType;
    char *cType;
    char *multiplyType;
    char *finalType;

    aType = getType("a");
    bType = getType("b");
    cType = getType("c");


    printf("Symbol Table\n");

    printf("-----------\n");


    printf("a : %s\n", aType);

    printf("b : %s\n", bType);

    printf("c : %s\n", cType);

    printf("\nExpression: a + b * c\n\n");


    printf("Type of a = %s\n", aType);

    printf("Type of b = %s\n", bType);

    printf("Type of c = %s\n", cType);

    multiplyType = resultType(bType, cType);

    printf("\nType of b * c = %s\n", multiplyType);

    finalType = resultType(aType, multiplyType);

    printf("Type of a + (b * c) = %s\n", finalType);


    printf("\nFinal expression type = %s\n", finalType);
```

```
    return 0;
}
```

Output :

```
PS E:\Compiler design\CO3-AT1> gcc Q2.c -o Q2.exe
PS E:\Compiler design\CO3-AT1> .\Q2.exe
Symbol Table
------------
a : int
b : int
c : float

Expression: a + b * c

Type of a = int
Type of b = int
Type of c = float

Type of b * c = float
Type of a + (b * c) = float

Final expression type = float
```

Q3. Write a C program to determine whether two type expressions are equivalent.

Code :

```c
#include <stdio.h>
#include <string.h>

struct Type
{
    char name[20];
    int size;
    char baseType[20];
};

int main()
{
    struct Type T1;
    struct Type T2;

    strcpy(T1.name, "array");
    T1.size = 10;
    strcpy(T1.baseType, "int");

    strcpy(T2.name, "array");
    T2.size = 10;
    strcpy(T2.baseType, "int");
```

```c
    printf("Type Expression 1: %s(%d, %s)\n",
           T1.name, T1.size, T1.baseType);

    printf("Type Expression 2: %s(%d, %s)\n",
           T2.name, T2.size, T2.baseType);

    if (strcmp(T1.name, T2.name) == 0 &&
        T1.size == T2.size &&
        strcmp(T1.baseType, T2.baseType) == 0)
    {
        printf("\nResult: Equivalent\n");
    }
    else
    {
        printf("\nResult: Not Equivalent\n");
    }

    return 0;
}
```

Output :

```
PS E:\Compiler design\CO3-AT1> gcc Q3.c -o Q3.exe
PS E:\Compiler design\CO3-AT1> .\Q3.exe
Type Expression 1: array(10, int)
Type Expression 2: array(10, int)

Result: Equivalent
```

Q4. Write a C program to construct and display the syntax tree for the following

Code :

```c
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    char data;
    struct Node *left;
    struct Node *right;
};

struct Node* createNode(char data)
{
```

```
    struct Node *newNode;

    newNode = (struct Node*)malloc(sizeof(struct Node));

    newNode->data = data;
    newNode->left = NULL;
    newNode->right = NULL;

    return newNode;
}

void displayTree(struct Node *root, int space)
{
    int i;

    if (root == NULL)
        return;

    space = space + 5;

    displayTree(root->right, space);

    printf("\n");

    for (i = 5; i < space; i++)
        printf(" ");

    printf("%c\n", root->data);

    displayTree(root->left, space);
}

int main()
{
    struct Node *root;
    struct Node *multiply;

    /* Create root '+' */
    root = createNode('+');

    /* Left side of '+' */
    root->left = createNode('a');

    /* Create '*' */
    multiply = createNode('*');

    /* Left and right of '*' */
    multiply->left = createNode('b');
```

```
    multiply->right = createNode('c');

    /* Right side of '+' */
    root->right = multiply;

    printf("Syntax Tree for: a + b * c\n");

    printf("\nTree:\n");

    displayTree(root, 0);

    return 0;
}
```

Output :

```
PS E:\Compiler design\CO3-AT1> gcc Q4.c -o Q4.exe
PS E:\Compiler design\CO3-AT1> .\Q4.exe
Syntax Tree for: a + b * c

Tree:

            c

      *

            b

+

      a
```

Q5. Write a C program to demonstrate the propagation of inherited and synthesized attributes using the following grammar:

Code :

```c
#include <stdio.h>
int main()
{
   char type[] = "int";

   char *id1 = "a";

   char *id2 = "b";

   char *id3 = "c";

   printf("Input: int a,b,c\n\n");

   printf("Grammar Processing:\n");

   printf("D -> T L\n");

   printf("T -> int\n");

   printf("L -> id L'\n");

   printf("L' -> , id L'\n");

   printf("L' -> epsilon\n");

   printf("\nAttribute Propagation:\n");

   printf("T.type = %s\n", type);

   printf("L.inherited_type = %s\n", type);

   printf("%s.type = %s\n", id1, type);

   printf("L'.inherited_type = %s\n", type);

   printf("%s.type = %s\n", id2, type);


   printf("L'.inherited_type = %s\n", type);
```

```
    printf("%s.type = %s\n", id3, type);

    printf("\nFinal Type Assignments:\n");

   printf("----------------------\n");

    printf("a : %s\n", type);

    printf("b : %s\n", type);

    printf("c : %s\n", type);

    return 0;
}
```

Output :

```
PS E:\Compiler design\CO3-AT1> gcc Q5.c -o Q5.exe
PS E:\Compiler design\CO3-AT1> .\Q5.exe
Input: int a,b,c

Grammar Processing:
D -> T L
T -> int
L -> id L'
L' -> , id L'
L' -> epsilon

Attribute Propagation:
T.type = int
L.inherited_type = int
a.type = int
L'.inherited_type = int
b.type = int
L'.inherited_type = int
c.type = int

Final Type Assignments:
----------------------
a : int
b : int
c : int
```